Practical 5

Build a small rule-based expert system or logic inference engine for a diagnosis or advisory problem.

Objective

To build a small rule-based expert system that performs forward-chaining inference over a knowledge base of symptom-to-diagnosis rules, and to use it to advise on a set of sample patient cases.

Procedure

1. Design a knowledge base of IF–THEN rules, each mapping a set of symptoms to a diagnosis and an advice string (e.g., fever + cough + sore_throat -> Common Cold).
2. Implement the DiagnosisExpertSystem class with an infer() method that performs forward chaining: a rule fires when its full symptom set is a subset of the patient's reported symptoms.
3. Prepare a set of test cases representing different patients with different combinations of reported symptoms.
4. Run the inference engine on each test case and record which rule(s), if any, fire.
5. Provide a default fallback advice message for cases where no rule matches confidently.
6. Report the diagnosis and advice generated for each patient case, along with the total number of rules in the knowledge base.

Program / Code

```
# ================================================================
# PRACTICAL 5
# Rule-Based Expert System - Medical Diagnosis Advisor
# ================================================================

# ---------------- Knowledge Base (Rules) ----------------
# Each rule: IF all symptoms present THEN diagnosis, with an advice string.

rules = [
    {
        "symptoms": {"fever", "cough", "sore_throat"},
        "diagnosis": "Common Cold / Flu",
        "advice": "Rest, fluids, and over-the-counter fever/cough medicine. See a doctor if symptoms
persist beyond 5-7 days."
    },
    {
        "symptoms": {"fever", "cough", "shortness_of_breath", "fatigue"},
        "diagnosis": "Possible Respiratory Infection",
        "advice": "Seek medical attention promptly; monitor oxygen levels if possible."
    },
    {
        "symptoms": {"headache", "nausea", "sensitivity_to_light"},
        "diagnosis": "Possible Migraine",
        "advice": "Rest in a dark quiet room; consult a doctor if this is a new or severe pattern."
```

```
    },
    {
        "symptoms": {"itchy_skin", "rash", "swelling"},
        "diagnosis": "Possible Allergic Reaction",
        "advice": "Avoid the suspected allergen; take an antihistamine; seek emergency care if
breathing is affected."
    },
    {
        "symptoms": {"fever", "joint_pain", "rash"},
        "diagnosis": "Possible Viral Infection (e.g., Dengue-like)",
        "advice": "Seek medical testing promptly; avoid aspirin/NSAIDs until confirmed."
    },
    {
        "symptoms": {"thirst", "frequent_urination", "fatigue"},
        "diagnosis": "Possible Diabetes Indicator",
        "advice": "Recommend a blood glucose test and consultation with a physician."
    },
]

DEFAULT_ADVICE = "Symptoms do not match a known rule confidently. Recommend consulting a physician for
proper evaluation."



class DiagnosisExpertSystem:
    """Simple forward-chaining rule-based inference engine."""

    def __init__(self, knowledge_base):
        self.rules = knowledge_base

    def infer(self, patient_symptoms):
        patient_symptoms = set(patient_symptoms)
        matches = []

        for rule in self.rules:
            # Forward chaining: fire the rule if ALL its symptom
            # conditions are satisfied by the patient's symptom set.
            if rule["symptoms"].issubset(patient_symptoms):
                matched_fraction = len(rule["symptoms"]) / len(rule["symptoms"])
                matches.append((rule, matched_fraction))

        return matches


if __name__ == "__main__":
    engine = DiagnosisExpertSystem(rules)

    test_cases = [
        {"fever", "cough", "sore_throat"},
        {"headache", "nausea", "sensitivity_to_light", "fatigue"},
        {"itchy_skin", "rash", "swelling"},
        {"fever", "cough", "shortness_of_breath", "fatigue"},
        {"thirst", "frequent_urination", "fatigue"},
        {"dizziness"},  # No matching rule
    ]

    print("=" * 65)
    print("       RULE-BASED EXPERT SYSTEM - MEDICAL DIAGNOSIS ADVISOR")
```

```
    print("=" * 65)

    for idx, symptoms in enumerate(test_cases, start=1):
        print(f"\nCase {idx}")
        print(f"Reported Symptoms : {sorted(symptoms)}")

        results = engine.infer(symptoms)

        if results:
            for rule, _ in results:
                print(f"  -> Diagnosis : {rule['diagnosis']}")
                print(f"     Advice    : {rule['advice']}")
        else:
            print(f"  -> Diagnosis : Unknown")
            print(f"     Advice    : {DEFAULT_ADVICE}")

        print("-" * 65)

    print("Total Rules in Knowledge Base :", len(rules))
    print("=" * 65)
```

Output

```
=================================================================
       RULE-BASED EXPERT SYSTEM - MEDICAL DIAGNOSIS ADVISOR
=================================================================

Case 1
Reported Symptoms : ['cough', 'fever', 'sore_throat']
  -> Diagnosis : Common Cold / Flu
     Advice    : Rest, fluids, and over-the-counter fever/cough medicine. See a doctor if symptoms
persist beyond 5-7 days.
-----------------------------------------------------------------

Case 2
Reported Symptoms : ['fatigue', 'headache', 'nausea', 'sensitivity_to_light']
  -> Diagnosis : Possible Migraine
     Advice    : Rest in a dark quiet room; consult a doctor if this is a new or severe pattern.
-----------------------------------------------------------------

Case 3
Reported Symptoms : ['itchy_skin', 'rash', 'swelling']
  -> Diagnosis : Possible Allergic Reaction
     Advice    : Avoid the suspected allergen; take an antihistamine; seek emergency care if breathing
is affected.
-----------------------------------------------------------------

Case 4
Reported Symptoms : ['cough', 'fatigue', 'fever', 'shortness_of_breath']
  -> Diagnosis : Possible Respiratory Infection
     Advice    : Seek medical attention promptly; monitor oxygen levels if possible.
-----------------------------------------------------------------

Case 5
Reported Symptoms : ['fatigue', 'frequent_urination', 'thirst']
  -> Diagnosis : Possible Diabetes Indicator
```

```
     Advice    : Recommend a blood glucose test and consultation with a physician.
------------------------------------------------------------------


Case 6
Reported Symptoms : ['dizziness']
  -> Diagnosis : Unknown
     Advice    : Symptoms do not match a known rule confidently. Recommend consulting a physician for
proper evaluation.
------------------------------------------------------------------
Total Rules in Knowledge Base : 6
==================================================================
```

Conclusion

A small forward-chaining rule-based expert system was implemented for a medical diagnosis-advisory task. Across six test cases, the engine correctly matched five patients to an appropriate rule (Common Cold, Migraine, Allergic Reaction, Respiratory Infection and Diabetes Indicator) purely by subset-matching their reported symptoms against the rule antecedents, and correctly fell back to a default 'consult a physician' recommendation for the one case (isolated dizziness) that matched no rule in the knowledge base. This demonstrates the core strength and limitation of rule-based systems: they give fast, explainable, deterministic advice when a case matches a known rule, but they cannot generalise to symptom combinations outside their hand-authored knowledge base, unlike a learned probabilistic model.